

Custom Animations (animate())

Concept:

Moving beyond basic show/hide effects to create smooth, custom CSS transitions.

Key Points:

- The syntax is `animate({ properties }, duration, callback)`.
- **The Golden Rule:** You can only animate *numeric* CSS values (like width: '200px', opacity: '0.5'). Core jQuery cannot animate colors directly.
- Always use camelCase for CSS properties in JavaScript (e.g., `fontSize` instead of `font-size`).

Task 1:

Move a box diagonally across the screen while simultaneously reducing its opacity.

Task 2:

Increase the font size and letter spacing of a heading when a button is clicked.

Solutions:

JavaScript

```
// Task 1 Solution: Moving diagonally
$("#btnMove").click(function(){
  $(".box").animate({
    left: '250px', // Requires CSS position: relative/absolute
    top: '250px',
    opacity: '0.5' // Fades to 50%
  }, 1000); // Takes 1 second
});
```

```
// Task 2 Solution: Typography animation
$("#btnText").click(function(){
  $("h1").animate({
    fontSize: '3rem', // Notice the camelCase
    letterSpacing: '10px'
  }, "slow");
});
```

Slide 2: Adding DOM Elements

Concept:

Injecting new HTML into the webpage dynamically using JavaScript.

Key Points:

- **Inside the Element:** `append()`(inserts at the very end) vs `prepend()`(inserts at the very beginning).
- **Outside the Element:** `after()`(inserts right after the closing tag) vs `before()`(inserts right before the opening tag).

Task 1:

Get the value from an input field and append it to the end of a `<ul>` as a new `<li>`.

Task 2:

Insert a "Processing..." `<div>` right above a form before it submits.

Solutions:

JavaScript

```
// Task 1 Solution: Append to a list
$("#addTodoBtn").click(function(){
  let task = $("#todoInput").val();
  if(task !== "") {
    // Inject HTML directly inside the UL
    $("#todoList").append("

```

- " + task + "

```
");  $("#todoInput").val(""); // Clear input field  } }); // Task 2 Solution: Insert before an element
```

```
$("#submitForm").click(function(){ // Adds a banner exactly before the
```

```
tag $("#myForm").before("

```

```
Processing your request...
```

```
"); });
```

Slide 3: Removing DOM Elements (remove vs empty)

Concept:

Cleaning up the UI by deleting elements to free up memory and prevent bugs.

Key Points:

- **empty():** The container stays where it is, but everything inside is thrown away.
- **remove():** This destroys the container entirely, along with everything inside it. Use this for dismissible alerts or deleted items.

Task 1:

Create a "Clear Cart" button that empties a `<ul>` list, but keeps the `<ul>` structure intact.

Task 2:

Completely delete a notification badge from the DOM when its "X" button is clicked.

Solutions:

JavaScript

```
// Task 1 Solution: Emptying a container
$("#clearCartBtn").click(function(){
  $("#cartItemsList").empty(); //
```

- tags vanish,

```
remains }); // Task 2 Solution: Completely removing an element $(".close-notification").click(function(){ // 'this' refers to
the specific button clicked. // .parent() finds its wrapper, and .remove() destroys it. $(this).parent().remove(); });
```

Slide 4: DOM Traversing

Concept:

Navigating up, down, and sideways through the DOM tree.

Key Points:

- When there are multiple identical elements (like 10 product cards), traversing allows you to target elements relative to the one that was clicked.
- Common methods: `parent()`, `children()`, and `find()`.

Task 1:

When a button is clicked, change the background color of *only* its direct parent `<div>`.

Task 2:

Find all `<span>` tags deeply nested inside a specific `<div id="section1">` and make them bold.

Solutions:

JavaScript

```
// Task 1 Solution: Targeting the parent
$(".highlight-btn").click(function(){
  $(this).parent().css("background-color", "yellow");
});

// Task 2 Solution: Finding descendants
$("#boldBtn").click(function(){
  // .find() searches all nested children
  $("#section1").find("span").css("font-weight", "bold");
});
```

Slide 5: Event Delegation (on())

Concept:

Attaching events to elements that don't exist yet on the page.

Key Points:

- Elements added to the page *after* the initial load (like dynamically appended To-Do items) will ignore standard `.click()` events.
- **Event Delegation:** We attach the event listener to a static parent element that already exists, and tell it to listen for clicks on its dynamic children using the `on()` method.

Task 1:

Toggle a "completed" CSS class on `<li>` items that were added dynamically via JavaScript.

Solution:

JavaScript

```
// Task 1 Solution: Event Delegation
// Target the static parent (#todoList), but listen for clicks on the dynamic child ('li')
$("#todoList").on("click", "li", function(){
    $(this).toggleClass("completed");
});
```